LAB REPORT

Doubly Linked List in Python

Addition, Insertion, Deletion, Searching, Updating and Basic Operations

Course	Data Structures and Algorithms Using Python
Lab No.	Lab 03
Student Name	Jiayu Yang （ID 20253801028）
Student ID	2025-AI-002
Submission Date	21 October 2026

1-1. Objective

The objective of this lab is to practice fundamental Doubly linked list operations in Python, including traversal, addition, insertion, deletion, searching, updating, aggregation, sorting and reversing. The lab also
demonstrates how these operations can be combined into a menu-driven Doubly Linked List Management System.

1-2. Software / Environment

Programming Language	Python 3.x
Editor / IDE	Editor
Data Structure	Python Doubly Linked list
File Name	20253801028_Doubly_Linked_List_operations.py

2-1. Doubly Listed List class definitions.

Doubly Linked List Class program

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def append(self, value):
        new = Node(value)
        if self.head is None:
            self.head = self.tail = new
        else:
            new.prev = self.tail
            self.tail.next = new
            self.tail = new

    def display_forward(self):
        current = self.head
        while current is not None:
            print(current.data, end=" <-> " if current.next else "")
            current = current.next
        print()
```

```
    def display_backward(self):
        current = self.tail
        while current is not None:
```

Doubly_Linked_List LAB#03 PYTHON

```
            print(current.data, end=" <-> " if current.prev else "")
            current = current.prev
        print()
```

```
    def prepend(self, value):
        new = Node(value)
        if self.head is None:
            self.head = self.tail = new
        else:
            new.next = self.head
            self.head.prev = new
            self.head = new

    def insert_at(self, index, value):
        if index < 0:
            return False
        if index == 0:
            self.prepend(value)
            return True
        current = self.head
        position = 0
        while current is not None and position < index:
            current = current.next
            position += 1
        if position != index:
            return False
        if current is None:
            self.append(value)
            return True
        new = Node(value)
        before = current.prev  #
        new.prev = before
        new.next = current
        before.next = new
        current.prev = new
        return True
```

Figure 1. Doubly Listed List class definitions

2-2. Doubly Listed List class definitions、DoublyLinkedList() and MAIN MENU.

Requirement: Place all indented methods above inside Doubly Linked List. Run the code below after the Node and Doubly Linked List class definitions. Then, into main_menu, 1. into

Python code

```
def main_menu():
    first = Node(10)
    second = Node(20)
    first.next = second
    second.prev = first
    print(first.next.data)   # 20
    print(second.prev.data)  # 10

    dll = DoublyLinkedList()
    for number in [10, 20, 30]:
        dll.append(number)
    dll.display_forward()   # 10 <-> 20 <-> 30
    dll.display_backward()  # 30 <-> 20 <-> 10

    dll.prepend(5)
    dll.display_forward()   #5 <-> 10 <-> 20 <-> 30
    dll.insert_at(2, 15)
    dll.display_forward()   #5 <-> 10 <-> 15 <-> 20 <-> 30

    dll.delete_first()
    dll.display_forward()   # 10 <-> 15 <-> 20 <-> 30
    dll.delete_last()
    dll.display_forward()   # 10 <-> 15 <-> 20
    print("searched index: ",dll.search(20))  #searched index: 2
    dll.update_at(2, 25)
    dll.display_forward()
    dll.delete_value(25)
    dll.display_forward()
    print("count: ",dll.count()) #count: 2
    dll.reverse()
    dll.display_forward()   # 30 <-> 25 <-> 10 <-> 5

    print(" 1. Entry Main Doubly Linked List_management")
    print(" 2. Remove precisely the selected node")
    print(" 3. Insert after the first matching node")
    print(" 4. Find the middle node")
    print(" 5. Remove duplicate values")
    while True:
        opt = input("\nPlease select task number: ")
        if opt == "1":
            main_list_management(dll)
        elif opt == "2":#challenge delete_at
            index = int(input("Enter index to delete: "))
            if dll.delete_at(index):
                dll.display_forward()
            else:
                print("delete at failed")
        elif opt == "3":   #challenge delete_at
            value = int(input("Enter find value: "))
            newValue = int(input("insert newValue: "))
            dll.insert_after(value,newValue)
            dll.display_forward()
        elif opt == "4":   #challenge find_middleNode
            dll.find_middleNode()
```

```
            dll.display_forward()
        elif opt == "5":   #challenge remove_duplicate
            dll.remove_duplicate()
            dll.display_forward()
```

```
PS C:\Users\36862\Desktop\lab3> python .\20253801028_Doubly_Linked_List_operations.py
20
10
10 <-> 20 <-> 30
30 <-> 20 <-> 10
5 <-> 10 <-> 20 <-> 30
5 <-> 10 <-> 15 <-> 20 <-> 30
10 <-> 15 <-> 20 <-> 30
10 <-> 15 <-> 20
searched index:  2
10 <-> 15 <-> 25
10 <-> 15
count:  (2, 1, 1)
15 <-> 10
 1. Entry Main Doubly Linked List_management
 2. Remove precisely the selected node
 3. Insert after the first matching node
 4. Find the middle node
 5. Remove duplicate values

Please select task number:
```

Figure 2. DoublyLinkedList() and main_menu

Action:select task number 1 into Doubly Linked List Management System,..... .

2-3. Doubly Linked List Management System

Function Doubly Linked List Management System task1-15

Python code

```
def main_list_management(dLinedList):

    print("DOUBLY LINKED LIST MANAGEMENT")
    print(" 1. Display Forward")
    print(" 2. Display Backward")
    print(" 3. Insert at Beginning")
    print(" 4. Insert at End")
    print(" 5. Insert at Index")
    print(" 6. Delete First")
    print(" 7. Delete Last")
    print(" 8. Delete by Value")
    print(" 9. Delete by Index")
    print("10. Search Value")
    print("11. Update by Index")
    print("12. Count Nodes")
    print("13. Sum / Min / Max / Average")
    print("14. Reverse List")
    print("15. Exit")
    while True:
        opt = input("\nPlease select task number: ")
        if opt == "1":
            dLinedList.display_forward()
        elif opt == "2":
            dLinedList.display_backward()
        elif opt == "3":
            val = int(input("Enter element at start: "))
            dLinedList.prepend(val)
            dLinedList.display_forward()
        elif opt == "4":
            val = int(input("Enter element at end: "))
            dLinedList.append(val)
            dLinedList.display_forward()
        elif opt == "5":
            index = int(input("Enter index: "))
            val = int(input("Enter element: "))
            if dLinedList.insert_at(index,val):
                dLinedList.display_forward()
            else:
                print("insert failed,index error")
        elif opt == "6":
            dLinedList.delete_first()
            dLinedList.display_forward()
        elif opt == "7":
            dLinedList.delete_last()
            dLinedList.display_forward()
        elif opt == "8":
            value = int(input("Enter value to delete: "))
            if dLinedList.delete_value(value):
                dLinedList.display_forward()
            else:
                print("delete at failed")
        elif opt == "9":
            index = int(input("Enter index to delete: "))
            if dLinedList.delete_at(index):
```

```
            dLinedList.display_forward()
        else:
            print("delete at failed")
    elif opt == "10":
        val = int(input("Enter element to search: "))
        if dLinedList.search(val) != -1:
           print("search value success")
        else:
           print("search value failed")
    elif opt == "11":
        index = int(input("Enter data to index: "))
        val = int(input("Enter element to add: "))
        if dLinedList.update_at(index,val):
            print("update value success")
        else:
            print("update value failed")
    elif opt == "12":
        dLinedList.display_forward()
        print("count_nodes:", dLinedList.count())
    elif opt == "13":
        dLinedList.display_forward()
        print("get_max value:", dLinedList.get_max())
        print("get_min value:", dLinedList.get_min())
        print("total_value:", dLinedList.get_sum())
        print("average_value:", dLinedList.get_average())
    elif opt == '14':
        dLinedList.display_forward()
        dLinedList.reverse()
        dLinedList.display_forward()
    elif opt == "15":
        print("Exit APP")
        break
    else:
        print("Invalid option, try again.")
```

```
Please select task number: 1
DOUBLY LINKED LIST MANAGEMENT
 1. Display Forward
 2. Display Backward
 3. Insert at Beginning
 4. Insert at End
 5. Insert at Index
 6. Delete First
 7. Delete Last
 8. Delete by Value
 9. Delete by Index
10. Search Value
11. Update by Index
12. Count Nodes
13. Sum / Min / Max / Average
14. Reverse List
15. Exit

Please select task number: 1
15 <-> 10

Please select task number: 2
10 <-> 15

Please select task number: 3
Enter element at start: 25
25 <-> 15 <-> 10

Please select task number: 4
Enter element at end: 35
25 <-> 15 <-> 10 <-> 35

Please select task number: 5
Enter index: 3
Enter element: 45
25 <-> 15 <-> 10 <-> 45 <-> 35

Please select task number: 6
15 <-> 10 <-> 45 <-> 35

Please select task number: 7
15 <-> 10 <-> 45

Please select task number: 8
Enter value to delete: 15
10 <-> 45

Please select task number: 9
Enter index to delete: 1
10

Please select task number:
```

```
Please select task number: 9
Enter index to delete: 1
10

Please select task number: 3
Enter element at start: 35
35 <-> 10

Please select task number: 4
Enter element at end: 34
35 <-> 10 <-> 34

Please select task number: 3
Enter element at start: 23
23 <-> 35 <-> 10 <-> 34

Please select task number: 3
Enter element at start: 23
23 <-> 23 <-> 35 <-> 10 <-> 34

Please select task number: 10
Enter element to search: 35
search value success

Please select task number: 11
Enter data to index: 1
Enter element to add: 32
update value success

Please select task number: 12
23 <-> 32 <-> 35 <-> 10 <-> 34
count_nodes: (5, 2, 3)

Please select task number: 13
23 <-> 32 <-> 35 <-> 10 <-> 34
get_max value: 35
get_min value: 10
total_value: 134
average_value: 26.8

Please select task number: 14
23 <-> 32 <-> 35 <-> 10 <-> 34
34 <-> 10 <-> 35 <-> 32 <-> 23

Please select task number:
```

Figure 2. Doubly Linked List Management System task1-15

3. Test Cases

For the definitions Doubly Linked List[10, 20, 30, 40, 60]

1. FUNCTION Task5 Insertion value 11, index 7, insert failed,index error
2. FUNCTION Task3 Insertion value 11, index 3, output [10, 20, 30,11, 40, 60]
3. FUNCTION Task5 Search value 50, search value failed

4. FUNCTION Task5 Search value 30, search value success

5. FUNCTION Task6 Update Index 1, value 55, update value success

```
Please select task number: 1
DOUBLY LINKED LIST MANAGEMENT
 1. Display Forward
 2. Display Backward
 3. Insert at Beginning
 4. Insert at End
 5. Insert at Index
 6. Delete First
 7. Delete Last
 8. Delete by Value
 9. Delete by Index
10. Search Value
11. Update by Index
12. Count Nodes
13. Sum / Min / Max / Average
14. Reverse List
15. Exit

Please select task number: 6
10

Please select task number: 6


Please select task number: 4
Enter element at end: 10
10

Please select task number: 4
Enter element at end: 20
10 <-> 20

Please select task number: 30
Invalid option, try again.

Please select task number: 4
Enter element at end: 30
10 <-> 20 <-> 30
```

```
Please select task number: 30
Invalid option, try again.

Please select task number: 4
Enter element at end: 30
10 <-> 20 <-> 30

Please select task number: 4
Enter element at end: 40
10 <-> 20 <-> 30 <-> 40

Please select task number: 4
Enter element at end: 60
10 <-> 20 <-> 30 <-> 40 <-> 60

Please select task number: 5
Enter index: 7
Enter element: 11
insert failed,index error

Please select task number: 5
Enter index: 3
Enter element: 11
10 <-> 20 <-> 30 <-> 11 <-> 40 <-> 60

Please select task number: 10
Enter element to search: 50
search value failed

Please select task number: 10
Enter element to search: 30
search value success

Please select task number: 11
Enter data to index: 1
Enter element to add: 55
update value success

Please select task number: 1
10 <-> 55 <-> 30 <-> 11 <-> 40 <-> 60

Please select task number:
```

Test	Operation	Input	Expected	Status
1	Insert	Value11 at Index 7	insert failed,index error	Pass
2	Insert	Value11 at Index 3	[10, 20, 30,11, 40, 60]	Pass
3	Search	50	search value failed	Pass
4	Search	30	search value success	Pass
5	Update	Index 1, Value 55	update value success	Pass

4. Lab tasks

attachment with 20253801028_Doubly_Linked_List_operations.py and Section. 2 Screenshots showing program execution

5. Advanced Task - Link Manipulation

Inserting a node N between two existing nodes A (before) and B (after) requires exactly four reference assignments:

```
N.prev = A
N.next = B
A.next = N
B.prev = N
```

Before: A <-> B (A.next = B, B.prev = A)

After: A <-> N <-> B (all four links above updated)

Missing any one of the four assignments leaves the chain broken: the new node would be unreachable from one side, or the neighbours would still point at each other and skip N.

Reversing the list swaps prev and next for every node. After the sweep, each node's old next (towards the tail) has become its prev, so the old head now points nowhere forward and the old tail points nowhere backward. Exchanging head and tail is therefore mandatory: the old tail is the new first node and the old head is the new last node. Without the exchange, head would reference the last node and forward traversal would print nothing useful.

Forward traversal after a correct reverse equals the previous backward traversal, which the program confirms with verify_links.